



گزارش کار جامع پروژه پایانی درس طراحی کامپایلر

پیاده‌سازی کامل فرانت‌اند کامپایلر و موتور هوشمند تحلیل کد IDE

دانشگاه: صنعتی خواجه نصیرالدین طوسی

دانشکده: مهندسی کامپیوتر

استاد درس: دکتر علاییان

سرپرست دستیاران (Head TA): علیرضا قربانی

نیمسال تحصیلی: بهار ۱۴۰۴-۱۴۰۵

اعضای تیم توسعه‌دهنده (دانشجویان):

آریا قزل ایاغ طهرانی

کوثر آموزگار

مبین فلاحی عشرت آبادی

زهرآ آقائی

۱. مقدمه و معماری کلی سیستم

پروژه پایانی درس طراحی کامپایلر با هدف پیاده‌سازی یک فرانت‌اند کامل کامپایلر به همراه سرویس‌های پیشرفته تحلیل ایستا (Static Analysis) و هوشمندی محیط توسعه (IDE Intelligence) طراحی شده است. این سیستم قابلیت تحلیل کامل کدهای ورودی زبان‌های برنامه‌نویسی (با پشتیبانی از ++C/C و گسترش‌پذیری افزونه‌ای)، استخراج ساختارهای نحوی و معنایی، ساخت گراف‌های کنترلی و فراخوانی، و ارائه قابلیت‌هایی نظیر Auto-completion، Hover info، Go-to-Definition، Rename Refactoring و کشف Dead Code را دارا می‌باشد.

تحلیل‌گر نحوی (Parser)

اعمال گرامر EBNF عاری از بازگشت به چپ و تولید درخت نحو انتزاعی (AST) غنی‌شده به همراه بازیابی خطا (Panic-Mode Recovery).

تحلیل‌گر لغوی (Lexer)

تبدیل جریان کاراکترهای ورودی به توکن‌های تایپ‌شده همراه با موقعیت دقیق (Line, Column) و پشتیبانی از قواعد Longest-match و Priority.

موتور تحلیل برنامه (Program Analyzer)

استخراج CFG، Call Graph، تحلیل Data-Flow، محاسبات Dominator Tree و تبدیل فرم SSA جهت بهینه‌سازی و بازسازی کد.

تحلیل‌گر معنایی (Semantic)

ایجاد جدول نمادهای چندسطحی (Scope Tree)، بررسی سیستم تایپ (Type Checking)، برقراری Bindings و استنتاج نوع.

۲. فاز اول: تحلیل لغوی، گرامر EBNF و Syntax Highlighting

۲.۱ تحلیل‌گر لغوی (Lexer) و اتوماتای حالت محدود (DFA)

ماژول [src/lexer.py](#) مسئولیت تبدیل جریان ورودی به توکن‌ها را بر عهده دارد. هر توکن شامل اطلاعات نوع توکن، مقادیر متنی (Lexeme) و موقعیت دقیق خط و ستون می‌باشد. تطبیق الگوها بر اساس عبارت‌های باقاعده (Regex) صورت گرفته و قاعده **Longest-Match (Maximal Munch)** و اولویت کلمات کلیدی به‌طور کامل رعایت شده است.

۲.۲ مشخصات گرامر (EBNF Grammar)

گرامر پروژه در فایل [grammar/grammar.ebnf](#) به فرم EBNF تعریف شده است. تمامی قوانین جهت جلوگیری از ابهام و امکان پیاده‌سازی به روش نزول بازگشتی (Recursive-Descent) اصلاح شده و بازگشت به چپ در آن حذف شده است.

```
program      ::= declaration* EOF
declaration  ::= function_decl | var_decl | struct_decl
function_decl ::= type_spec IDENT '(' param_list? ')' block
param_list   ::= param (',' param)*
param        ::= type_spec IDENT
type_spec    ::= ('int' | 'float' | 'char' | 'void' | 'double') '*'*
block        ::= '{' statement* '}'
statement    ::= if_stmt | while_stmt | for_stmt | return_stmt | expr_stmt | block | var_decl
if_stmt      ::= 'if' '(' expr ')' statement ('else' statement)?
return_stmt  ::= 'return' expr? ';'
expr         ::= assignment
assignment   ::= IDENT ('=' | '+=' | '-=' | '*=') assignment | logical_or
```

```
additive ::= multiplicative (('+' | '-') multiplicative)*
primary  ::= INT | FLOAT | STRING | CHAR | IDENT | '(' expr ')'
```

۲.۳ رنگ‌آمیزی سنتکس مبتنی بر AST

سیستم پیاده‌سازی شده در `src/highlighter.py` با پیمایش AST و استعلام از جدول نمادها عمل هایلایت را انجام می‌دهد که تمایز دقیق بین نام تابع و متغیر عادی را ممکن می‌سازد.

۳. فاز دوم: تحلیل معنایی و موتور هوشمند Intellisense

۳.۱ ساختار جدول نمادها (Symbol Table) و مدیریت Scope

جدول نمادها در `src/semantic.py` به صورت یک ساختار درختی (Scope Tree) پیاده‌سازی شده است که ساختار بلوکی برنامه را منعکس می‌کند.

۳.۲ سیستم بررسی و استنتاج نوع (Type System)

در زبان‌های استاتیک، تعاریف انواع و تطبیق نوع در انتساب‌ها، فراخوانی توابع و عملگرها بررسی می‌شود. برای زبان‌های پویا نظیر پایتون، استنتاج نوع جریان‌محور (Flow-Sensitive Type Inference) اعمال می‌شود.

۳.۳ موتور تکمیل خودکار کد و Diagnostic System

موتور تکمیل خودکار بر اساس کنتکست تکمیلی، نمادهای معتبر در موقعیت مکان‌نما را استعلام کرده و پیشنهاد می‌دهد. همچنین خطاهای معنایی نظیر `Undefined Symbol` و `Type Mismatch` به همراه محدوده دقیق گزارش می‌شوند.

۴. فاز سوم: تحلیل پیشرفته برنامه، Call Graph، CFG، Refactoring

۴.۱ ساخت گراف جریان کنترل (CFG) و گراف فراخوانی (Call Graph)

در ماژول `src/program_analysis.py`، برای هر تابع یک CFG تشکیل می‌شود که شامل Basic Block ها و مسیرهای اجرای ممکن است. همچنین Call Graph برنامه برای تحلیل فراخوانی‌های بین توابعی محاسبه می‌گردد.

۴.۲ تحلیل‌های جریان داده (Data-Flow Analysis)

- **Definite Assignment Analysis**: اطمینان از مقداردهی قطعی متغیر قبل از خوانده شدن.
- **Live Variable Analysis**: تحلیل پس‌رو جهت شناسایی متغیرهای زنده و انتساب‌های مرده.
- **Unreachable Code Detection**: شناسایی کدهای غیرقابل دسترس و دستورات پس از jump.

۵. ویژگی‌های پیشرفته و بخش‌های امتیازی (Bonus Features)

در راستای ارتقای کیفیت فنی پروژه، امکانات فوق‌العاده‌ای پیاده‌سازی شده است:

- **پیش‌پردازنده و بازکردن ماکروها**: پشتیبانی از دستورات `define#` در `src/preprocessor.py`.
- **پشتیبانی چند زبانه (افزونه پایتون)**: تحلیل کد پایتون در `src/python_plugin.py`.
- **تحلیل Dominator Tree و SSA Form**: پیاده‌سازی کامل درخت تسلط و تبدیل فرم Static Single Assignment.
- **پارسر افزایشی (Incremental Parsing)**: به‌روزرسانی زنده AST بر اساس تغییرات منطقه خطوط.
- **تخمین خودکار زبان برنامه (Language Detection)**: شناسایی خودکار زبان بر اساس کلمات کلیدی، نشانه‌ها و ساختار.
- **تولید کد سه‌آدرسه (TAC / IR Code)**: تولید کدهای واسط کم‌سطح جهت بهینه‌سازی.
- **استقرار با Docker و CI/CD GitHub Actions**: ساخت کانتینر خودکار و تست‌های واحد پیوسته.

۶. راهنمای نصب و نحوه اجرای پروژه

بر اساس مستندات پروژه، مراحل اجرای سیستم به شرح زیر است:

۶.۱ پیش‌نیازها و نصب وابستگی‌ها

```
pip install -r requirements.txt
```

۶.۲ اجرای کامپایلر از طریق خط فرمان (CLI)

```
python main.py tests/test_code.c
```

۶.۳ اجرای محیط تعاملی وب (Web UI)

```
python web_ui.py
```

سپس در مرورگر خود به آدرس <http://127.0.0.1:5000> مراجعه کنید.

۶.۴ اجرا با استفاده از Docker

```
# ساخت تصویر داکر
docker build -t compiler-project .

# اجرای کانتینر
docker run -p 5000:5000 compiler-project
```

۶.۵ اجرای تست‌های خودکار و گزارش پوشش کد (Coverage)

```
pytest tests/
pytest --cov=src tests/
```

۷. بررسی خروجی دکمه‌های محیط وب (Web UI) و تصویر عملکردها

در این بخش، خروجی‌های تولید شده توسط دکمه‌های مختلف رابط کاربری وب ([web_ui.py](#)) به همراه تصاویر مربوطه شرح داده شده است:

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int x = 10;  
    int y = 20;  
    int z = add(x, y);  
    int d = 100;  
  
    if (x > 5) {  
        y = 30;  
    } else {  
        y = 40;  
    }  
  
    return y;  
    x = 50;  
}
```

Program Analysis & Optimization (Bonus Features)

- Expand Macros
- Parse Python (Plugin)
- Dominator Tree
- Reaching Definitions
- SSA Form (φ)
- Incremental Parse
- Detect Dead Code
- Eliminate Dead Code (DCE)
- Generate TAC (IR Code)
- Data Flow
- Call Graph (JSON)
- Visual Call Graph
- Visual AST Diagram
- Detect Language

Symbol:

e.g. x

Line:

2

New Name:

New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

Results will appear here...

شکل ۱: نمای کلی محیط تعاملی Web UI پروژه -شامل ویرایشگر کد منبع، پنل دکمه‌های تحلیلی و کنسول خروجی.

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int x = 10;  
    int y = 20;  
    int z = add(x, y);  
    int d = 100;  
  
    if (x > 5) {  
        y = 30;  
    } else {  
        y = 40;  
    }  
  
    return y;  
    x = 50;  
}
```

Program Analysis & Optimization (Bonus Features)

- Expand Macros
- Parse Python (Plugin)
- Dominator Tree
- Reaching Definitions
- SSA Form (φ)
- Incremental Parse
- Detect Dead Code
- Eliminate Dead Code (DCE)
- Generate TAC (IR Code)
- Data Flow
- Call Graph (JSON)
- Visual Call Graph
- Visual AST Diagram
- Detect Language

Symbol:

add

Line:

1

New Name:

New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

[FUNCTION] add(int, int) -> int
Defined at line 1, scope: Global

شکل ۲: خروجی دکمه Hover Info - نمایش اطلاعات نوع، امضای تابع و اسکوپ تعریف نماد انتخابی (مثلاً نماد add).

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int x = 10;  
    int y = 20;  
    int z = add(x, y);  
    int d = 100;  
  
    if (x > 5) {  
        y = 30;  
    } else {  
        y = 40;  
    }  
  
    return y;  
    x = 50;  
}
```

Program Analysis & Optimization (Bonus Features)

Expand Macros Parse Python (Plugin) Dominator Tree Reaching Definitions
SSA Form (φ) Incremental Parse Detect Dead Code Eliminate Dead Code (DCE)
Generate TAC (IR Code) Data Flow Call Graph (JSON) Visual Call Graph
Visual AST Diagram Detect Language

Symbol:

add

Line:

1

New Name:

New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

```
{  
  "symbol": "add",  
  "kind": "function",  
  "type": "int(int, int)",  
  "defined_at": {  
    "file": "web_editor.c",  
    "line": 1,  
    "column": 1  
  },  
  "references": [  
    {  
      "file": "web_editor.c",  
      "line": 8,  
      "col": 13  
    }  
  ]  
}
```

شکل ۳: خروجی دکمه Go to Definition - بازگرداندن فایل، خط، ستون محل تعریف و تمامی ارجاعات به نماد به صورت ساختار JSON.

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int x = 10;  
    int y = 20;  
    int z = add(x, y);  
    int d = 100;  
  
    if (x > 5) {  
        y = 30;  
    } else {  
        y = 40;  
    }  
  
    return y;  
    x = 50;  
}
```

Program Analysis & Optimization (Bonus Features)

Expand Macros Parse Python (Plugin) Dominator Tree Reaching Definitions
SSA Form (φ) Incremental Parse Detect Dead Code Eliminate Dead Code (DCE)
Generate TAC (IR Code) Data Flow Call Graph (JSON) Visual Call Graph
Visual AST Diagram Detect Language

Symbol:

add

Line:

1

New Name:

New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

```
UNUSED VARIABLE: 'z' declared at line 8 is never read  
UNUSED VARIABLE: 'd' declared at line 9 is never read  
UNREACHABLE CODE: Statement at line 18 is dead code (after return/break/continue)
```

شکل ۴: خروجی دکمه Detect Dead Code - شناسایی دقیق متغیرهای خوانده نشده (Unused Variables) و دستورات غیرقابل دسترس پس از return.

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 10;
    int y = 20;
    int z = add(x, y);
    int d = 100;

    if (x > 5) {
        y = 30;
    } else {
        y = 40;
    }

    return y;
    x = 50;
}
```

Program Analysis & Optimization (Bonus Features)

Expand Macros

Parse Python (Plugin)

Dominator Tree

Reaching Definitions

SSA Form (Φ)

Incremental Parse

Detect Dead Code

Eliminate Dead Code (DCE)

Generate TAC (IR Code)

Data Flow

Call Graph (JSON)

Visual Call Graph

Visual AST Diagram

Detect Language

Symbol: add

Line: 1

New Name: New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

Detected Language: c_cpp

Confidence: 58.02%

Scores Breakdown:

- bash: 0%

- c_cpp: 58.02%

- java: 20.99%

- javascript: 20.99%

- python: 0%

شکل ۵: خروجی دکمه Detect Language - تشخیص خودکار زبان برنامه‌نویسی کد ورودی همراه با درصد اطمینان (Confidence) و تفکیک امتیاز زبان‌ها.

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 10;
    int y = 20;
    int z = add(x, y);
    int d = 100;

    if (x > 5) {
        y = 30;
    } else {
        y = 40;
    }

    return y;
    x = 50;
}
```

Program Analysis & Optimization (Bonus Features)

Expand Macros

Parse Python (Plugin)

Dominator Tree

Reaching Definitions

SSA Form (Φ)

Incremental Parse

Detect Dead Code

Eliminate Dead Code (DCE)

Generate TAC (IR Code)

Data Flow

Call Graph (JSON)

Visual Call Graph

Visual AST Diagram

Detect Language

Symbol: add

Line: 1

New Name: New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

main

↓

add

شکل ۶: خروجی دکمه Visual Call Graph - رسم گراف تعاملی فراخوانی توابع برنامه (ارتباط بین main و add) با استفاده از Mermaid.js.

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 10;
    int y = 20;
    int z = add(x, y);
    int d = 100;

    if (x > 5) {
        y = 30;
    } else {
        y = 40;
    }

    return y;
    x = 50;
}
```

Program Analysis & Optimization (Bonus Features)

- Expand Macros
- Parse Python (Plugin)
- Dominator Tree
- Reaching Definitions
- SSA Form (ϕ)
- Incremental Parse
- Detect Dead Code
- Eliminate Dead Code (DCE)
- Generate TAC (IR Code)
- Data Flow
- Call Graph (JSON)
- Visual Call Graph
- Visual AST Diagram
- Detect Language

Symbol:

add

Line:

1

New Name:

New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations



شکل ۷: خروجی دکمه Visual AST Diagram - رسم دیاگرام بصری درختی نحو انتزاعی (AST) به صورت گرافیکی و تعاملی.

Compiler Advanced IDE (Ultimate Bonus Package)

Source Code

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 10;
    int y = 20;
    int z = add(x, y);
    int d = 100;

    if (x > 5) {
        y = 30;
    } else {
        y = 40;
    }

    return y;
    x = 50;
}
```

Program Analysis & Optimization (Bonus Features)

- Expand Macros
- Parse Python (Plugin)
- Dominator Tree
- Reaching Definitions
- SSA Form (ϕ)
- Incremental Parse
- Detect Dead Code
- Eliminate Dead Code (DCE)
- Generate TAC (IR Code)
- Data Flow
- Call Graph (JSON)
- Visual Call Graph
- Visual AST Diagram
- Detect Language

Symbol:

add

Line:

1

New Name:

New name

Hover Info

Go to Definition

Rename Refactor

Output Console & Visualizations

=== CONTROL FLOW GRAPH: DOMINATOR TREE ANALYSIS ===

- ★ Dominator Sets ($Dom(n)$):
 - $Dom(ENTRY) = \{ ENTRY \}$
 - $Dom(B1 \text{ (evaluate condition)}) = \{ ENTRY, B1 \}$
 - $Dom(B2 \text{ (true branch)}) = \{ ENTRY, B1, B2 \}$
 - $Dom(B3 \text{ (false branch)}) = \{ ENTRY, B1, B3 \}$
 - $Dom(EXIT) = \{ ENTRY, B1, EXIT \}$
- ★ Immediate Dominators ($IDom(n)$):
 - $IDom(B1 \text{ (evaluate condition)}) \rightarrow ENTRY$
 - $IDom(B2 \text{ (true branch)}) \rightarrow B1 \text{ (evaluate condition)}$
 - $IDom(B3 \text{ (false branch)}) \rightarrow B1 \text{ (evaluate condition)}$
 - $IDom(EXIT) \rightarrow B1 \text{ (evaluate condition)}$

شکل ۸: خروجی دکمه Dominator Tree - محاسبه و نمایش مجموعه‌های تسلط $Dom(n)$ و Immediate Dominators $IDom(n)$ برای بلوک‌های گراف جریان کنترل.

پروژه پیاده‌سازی شده توسط تیم توسعه، تمامی نیازهای یک فرانت‌اند کامل کامپایلر و محیط هوشمند تحلیل کد را پوشش داده و ابزاری کاملاً کارآمد، تعاملی و عملیاتی را به ارمغان آورده است.

گزارش کار جامع پروژه پایانی درس طراحی کامپایلر - دانشگاه صنعتی خواجه نصیرالدین طوسی

استاد درس: دکتر علاییان | بهار ۱۴۰۴-۱۴۰۵